

Iress Web Services V4

Programmers Guide



Contents

Iress Web Services Overview	5
Overview	5
Architecture	6
End Points	7
Getting Started	8
Object & Parameter Descriptions	12
Requests	12
Responses	13
Session Management	14
Iress Sessions	15
Starting an Iress session	15
Ending an Iress session	15
Service Sessions	16
Starting a service session	16
Service session lifetime	16
Ending a service session	16
User Scenarios	17
Scenario 1: List out the sessions in use	17
Scenario 2: Force close any existing session	19
Session Error Management & Session Recovery	22
Session Error Management	22
Error handling and mitigation	22
Best-practice workflow	23
Recovery	24
Session Expiration	24
Paging	26
Overview	26
Paging with bookmarks	26
Paging in IPS	28
Updates	32
Synchronous vs Asynchronous Calling	34
API Methods by Application	35
Methods by Product	35
Iress Pro	36
Market Data and Web Admin	36

IRESSSessionStart	36
TimeSeriesGet2	36
PricingQuoteGet	36
IOS+	37
ServiceSessionStart for IOS+ service	37
Create order (OrderCreate3)	37
Amend order (OrderAmend2)	37
Cancel order (OrderDelete)	37
OrderPadGetByAccount and OrderPadGetByAccountUpdates	37
Get bookings (BookingGetByOrganisation2)	38
Explanation of XML fields in <MiscFees> tags	38
Get ETCs (ETCGetByOrganisation)	40
IOS+ Contingent Orders	41
Contingent order destinations	41
Order attributes – Iress strategy properties	41
FIXED CO	42
TRAILING CO	43
OCO	44
IFDONE	44
Order placement with take profit	45
IOS+ External Algo Orders	46
Algo destinations	46
Order attributes – External algo properties	48
IPS	51
ServiceSessionStart for IPS service	51
Get IPS transactions (IPSTransactionGetByAccount5)	51
IPS uploads	52
IPSUploadCreate1	52
IPSUploadDataSet1	52
IPSUploadRun1	53
IPSUploadSummaryGet2	53
IPSUploadErrorGet1	53
FIX+	54
ServiceSessionStart for FIX+ service	54
TargetIDGet	54
TargetIDStatusGet	54
Application Error Management	55

Application Recovery	57
Iress WebAdmin methods	57
IOS+ admin methods	57
IOS+ order retrieval	58
IOS+ order creation	59
Client-side Optimisations and Server-side Limits	61
Client-side optimisations	61
Server-side limits	61
Backward Compatibility	63
WSDL compatibility	63
Appendix	65
Possible session error codes and causes	65
Support queries	70
Frequently Asked Questions	72
Request has exceeded the maximum of 10000 queued updates	72
The session has reached the limit of 50 active requests. You cannot create any more requests	72
Do you support compression?	72
API Method Quick Reference	74
Iress Pro	74
IOS+	74
Get ETCs (ETCGetByOrganisation)	75
IOS+ Contingent Orders	75
FIX+	75
IPS	76

Iress Web Services Overview

Throughout this document you can find links to samples of SOAP XML requests and responses. All of the Web Services V4 samples linked in this document are available at this public GitHub repo:

<https://github.com/iress/webservices-v4-docs>.



The samples include examples of SOAP XML requests and responses for key methods across each application, as well as a handful in different programming languages.

Overview

Web Services V4 is a SOAP API that provides a platform-agnostic solution to allow access to market data, trading and portfolio functionality. It offers a platform-independent, scalable, and redundant server-based solution without the need of running the Iress front-end software. This version is a Document/Literal style web service, designed for modern SOAP toolkits, such as .NET and Java 6.

The current flow is to connect to the Web server using soap requests over Http. The web server is responsible for holding the Http connection for the soap request and responding with the appropriate data. The web server translates the received soap XML payload to and from Iress systems.

Use this version to access:

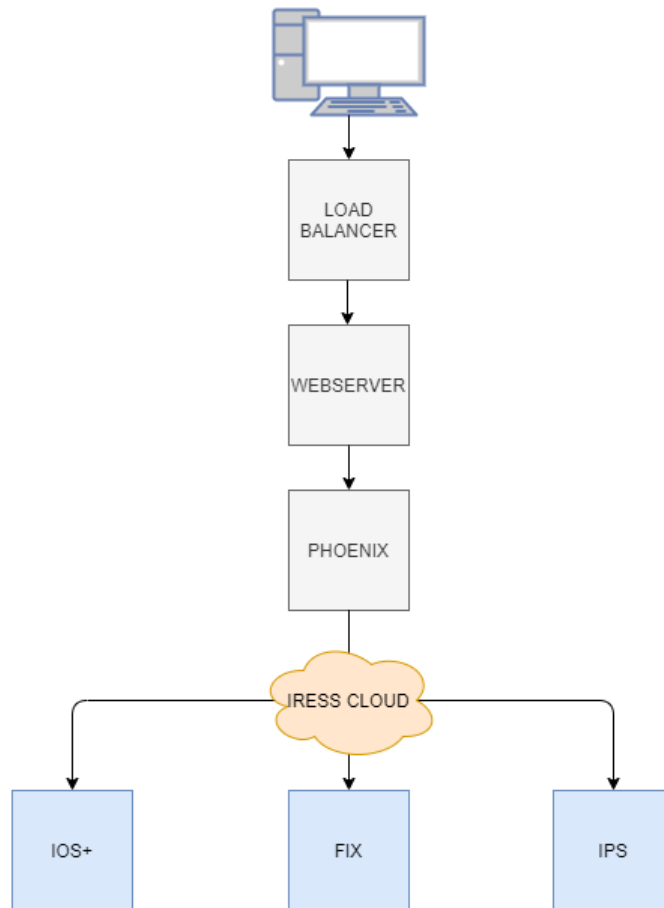
- Iress
- Portfolio System (IPS)
- Order System (IOS+)
- FIX+

This version of Web Services provides the ability to:

- Watch for updates via a long polling mechanism
- Page through large result sets
- Execute asynchronously

You can access Web Services either via our central web servers (see section 3 for public endpoints), or for clients who have their own local Phoenix server setup, we can also set up Web Services locally. Web Services requires users to hold the permission "Web Services" at the group level in Iress Web Administration.

Architecture



End Points

Type	URL	Environment
Australia Production	https://webservices4.iress.com.au	Production
	https://webservices.iress.com.au/v4/	
Australia Production Test	https://webservicestesta.iress.com.au/v4/	ASX FTE
	https://webservicestestb.iress.com.au/v4/	ASX ETE
	https://webservicestestc.iress.com.au/v4/	ASX FTE
Canada Production	https://webservices.iress.ca/v4/	
Canada Production Test	https://webservicetest.iress.ca/v4/	
Singapore Production	https://webservices.iress.com.sg/v4/	
Singapore Production Test	https://webservicetest.iress.com.sg/v4/	
South Africa Production	https://webservices.iress.co.za/v4	
South Africa Production test	https://webservices-ct.iress.co.za/v4	
UK Production	https://webservices.iress.co.uk/v4	
UK Production Test	https://webservicetest.iress.co.uk/v4	

Getting Started

To get started with accessing web services, you first need to find a URL to call the API.

Navigate to the web service URL you are provided with, using the [End Points](#) table, to get the WSDL form. For example, for the AU region, the public test environment URL would be <https://webservices testa.iress.com.au/v4/> . You can refer to Section 3 to get the relevant URLs for your region.

Open the URL and click **WSDL** to enter your credentials.

You will need to provide:

- Service you wish to use (IRESS, IOS+, IPS, FIX+)
- The name of the server you wish to connect to (for IOS+, IPS and FIX+ services only)
- Iress username
- Iress company name
- Iress password
- Method filter (optional)



Method filters are useful when you do not want to download all methods. Instead, you can download the ones you are interested in by separating them with commas. By doing this, you can make the overall development environment and method calling much faster.

IRESS Web Services Version 4

[Home](#)

Introduction

This is the latest generation of web service interface, for accessing the newest IRESS systems. This version

Use this version to access:

- IRESS
- IPS
- IOSPlus
- FIXPlus

Documentation

[WSDL](#)

[Method Reference](#)

External documentation

[Programmer's Guide](#)

[Send feedback](#) on this page.

Copyright (C) 2009 IRESS Market Technology

IRESS Web Services Version 4

[Home](#)

WSDL

Use this form to generate the WSDL. You will need an IRESS user name and password. Contact your IRESS account executive if you don't have one.

 This operation will temporarily use your IRESS licence while it is running.

Service:	IOSPlus
Server:	AEDEMOP
Authentication: 	HTTP Basic
Username @ Company:	@
Password:	
Method Filter: 	OrderCreate3
WSDL	

[Send feedback](#) on this page.

Copyright (C) 2009 IRESS Market Technology

When you access the WSDL, it will validate the provided credentials. If the validation succeeds then the WSDL will be dynamically generated.

[←](#) [→](#) [↻](#) [🔒](#) websericestesta.iress.com.au/v4/wsdl.aspx?svc=IOSPlus&svr=AEDEMOP&mf=OrderCreate3

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version='1.0' encoding='utf-8'>
<wsdl:definitions xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:tm="http://microsoft.com/wsdl/mime/textMatching/" xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/" xmlns:
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:soap12="http://schemas.xmlsoap.org/wsdl/soap12/" xmlns:http="http://schemas.xmlsoap.org/wsdl/http/" xmlns:tns="http://webservices.iress.c
targetNamespace="http://webservices.iress.com.au/v4/">
  <wsdl:types>
    <xsd:schema xmlns:tns="http://webservices.iress.com.au/v4/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" targetNamespace="http://webservices.iress.com.au/v4/" elementFormDefault="qualifi
      <xsd:complexType name="IRESSFaultDetail">
        <xsd:sequence>
          <xsd:element minOccurs="1" maxOccurs="1" name="Message" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="Number" nillable="true" type="xsd:int"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="Context" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="Service" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="Server" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="Method" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="UserName" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="CompanyName" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="EndPoint" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="RequestId" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="SessionKey" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="ServiceSessionKey" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="WebServiceTimeStamp" nillable="true" type="xsd:dateTime"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="WebServiceServer" nillable="true" type="xsd:string"/>
          <xsd:element minOccurs="1" maxOccurs="1" name="WebServiceConnection" nillable="true" type="xsd:string"/>
        </xsd:sequence>
        <xsd:complexType base="xsd:string" name="Input">
          <xsd:sequence>
            <xsd:any processContents="skip"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
    </xsd:sequence>
  </xsd:complexType>
  <xsd:element name="IRESSFaultDetail" type="tns:IRESSFaultDetail"/>
  <xsd:complexType name="IRESSDictionary">
    <xsd:annotation>
      <xsd:appinfo>
        <IsDictionary xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</IsDictionary>
      </xsd:appinfo>
    </xsd:annotation>
    <xsd:sequence>
      <xsd:element minOccurs="0" maxOccurs="unbounded" name="Elements">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="Key" nillable="true" type="xsd:string"/>
            <xsd:element name="Value" nillable="true" type="xsd:string"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
    </xsd:sequence>
  </xsd:complexType>

```

Using the above you can do the following:

- Create an Iress session
- Create a service session (if calling methods against IOS+, IPS or FIX+ services)
- Call desired methods
- Process response or handle error for each method
- End the Service Session (if calling IOS+, IPS or FIX+) and end the Iress Session

After you add your service/web reference, you can make a  [request](#).

If a  request executes normally, you will receive a  response **Output**; otherwise, you will receive a **SOAP fault**.

Object & Parameter Descriptions

Requests

There are two kinds of requests:

Data Request: A **Data Request** allows you to retrieve historical/snapshot-based data or send data. They also allow you to subscribe for updates if it is supported by the request.

- **Iress Request:** Iress Market Data
- **Service Request:** IOS+, IPS and FIX+

When requesting for updates, the **Data Request** and the **Updates Request** are "linked" by using the same **RequestID** in the Input header.

Updates Request: An **Updates Request** allows you to subscribe for updates and retrieve those updates.

 For more information on handling live updates, see [Updates](#).

The request Input has two components:

- **Header:** This differs slightly between an Iress session and a Service session (IOS+, IPS, FIX+).

SessionKey: The key you obtain from calling *IRESSSessionStart* method. (For Iress requests)

ServiceSessionKey: The key you obtain from calling the *ServiceSessionStart* method. (For IOS+, IPS, FIX+)

WaitForResponse: Whether the request blocks while waiting for data to come back from the data server, or have the web server return an empty response with **StatusCode** of 1 indicating there is more data available in which the user can then poll the server for a response. The default is True. This parameter determines whether your method call is asynchronous or synchronous.

PageSize: The maximum number of rows per page. It defaults to 1000 unless overridden by the data server. The data server may impose a maximum page size that is less than the value provided.

RequestID: The unique identifier of the request. This will be used for identifying the request when paging or using updates. It is recommended that this be set to a globally unique identifier (GUID).

Timeout: The time in seconds that the request waits to receive a reply from the data server. The default is 25 seconds unless overridden by the data server. The data server may impose a shorter or longer timeout than what is specified here e.g., *IRESSSessionStart* requests use a 55-second timeout

Updates: Whether to start watching for updates if the method supports updates. The default is false. If the method does not support updates, this parameter is ignored.

PagingBookmark: The bookmark field if the method supports server-side paging.

PagingDirection: The default is forward. Backwards is not implemented.

- **Parameters:** Each method has its own set of parameters (if any).

Responses

If a request is executed without error, you will receive an output; otherwise, you will receive a SOAP fault.

A response output has the following components:

- **Input:** This is the input, echoed back. It is on by default but can be configured at the web server level if the client has their own local web server setup
- **Result:** This contains the response data. It has the following components:
 - Header:** A response header. This header has the following components:
- **RequestID:** A unique identifier that is echoed back (if you provided one.) If you didn't provide an identifier, the server will automatically generate one for you.
- **StatusCode:** A number indicating the status of the request. Possible values are:

1. **More data available:** Call again with the same RequestID to get the data.
2. **Finished:** The request has ended.
3. **Watching for updates:** The request is watching for updates. Call the updates method to retrieve any queued updates.

HeaderRow: A row of header data that is specific to the request returned from the server.

DataRows: An array of rows of data that is specific to the request returned from the server.

ErrorRows: An array of rows of errors that is specific to the request returned from the server.

Session Management

📄 Go [here](#) for more advanced session management scenarios. Or [here](#) for information on managing errors and session recovery.

You can create two types of Iress session:

- [Iress Session](#)
- [Service Session](#)

The very first thing you should do is obtain an **IRESSSessionKey** by [calling the *IRESSSessionStart* method](#). The key acts as a token that tells the system who you are and what operations you are allowed to execute.

An Iress session allows you to:

- Create Service sessions (IOS+, IPS and FIX+)
- Call Iress Market Data methods

An Iress session will, by default, expire after two hours of inactivity. The session lifetime counter starts after your last request finishes and will restart if you perform another request through that same session. When a session expires, you can't use it anymore. Any pending data will be deleted. You will have to establish another session if you want to make more requests.

The maximum possible lifetime of an Iress session is 24 hours.

Iress Sessions

Starting an Iress session

To establish an Iress session and obtain a session key, you call the *IRESSSessionStart* method. The method takes the following parameters:

- **UserName:** Iress user name
- **CompanyName:** Iress company name
- **Password:** Iress password
- **ApplicationID:** This should be a unique value each time *IRESSSessionStart* is called. As an example, this could be the name of your application in combination with a GUID.
- **ApplicationLabel:** An optional additional marker to identify your type of application.

To create another Iress session, you need to pass a unique **ApplicationID** along with **UserName** and **CompanyName**.

Each new Iress session (defined by an **IRESSSessionKey**) requires the use of one license. If you need to use a high number of concurrent processes, please contact your Iress Account Executive and request they set the license count on the given user login to a high number. Be aware that this may attract additional license costs but will prevent the “No more licenses available” error.

We recommend always managing your active sessions by logging out or following one of the two scenarios described in 7.2. You can ensure you’re logged out entirely by first ending any IOS+, IPS or FIX+ Service Session(s) and then the Iress session.



The number of concurrent method calls that can be active at any time is 50 per **IRESSSessionKey**. You need to ensure your session management handling does not breach this limit.

Ending an Iress session

After you finish using the session, you can explicitly terminate the session and free your login count that the session was using. To end an Iress session, you call the *IRESSSessionEnd* method. This method does not have any parameters, but you will need to set the **IRESSSessionKey** in the header.

Alternatively, you could let your session expire, however ending an Iress session will give you some statistics about your session, allowing a session to expire will not.

Service Sessions

To request data from IOS+, IPS and FIX+, you need to establish an active service session with the server you wish to retrieve data from and obtain a **ServiceSessionKey**.

Starting a service session

To obtain a **ServiceSessionKey**, you call the *ServiceSessionStart* web method. The method takes the following parameters:

- **IRESSSessionKey**: The key you obtained when you called the *IRESSSessionStart* method
- **Service**: The service type, i.e. "IOSPlus", "IPS" or "FIXPlus"
- **Server**: The name of the server to connect to, e.g. "IOSPLUSAPI"

If the method is successful, you will get the following:

- **ServiceSessionKey**: The token you will have to use to call service-based methods

Service session lifetime

A service session expires with its parent Iress session.

Ending a service session

There are two ways to terminate a service session, directly and indirectly.

1. To terminate the session directly, you call the *ServiceSessionEnd* method. It does not take any parameters but you need to set the **ServiceSessionKey** in the header.
2. To terminate all service sessions associated with a single Iress session, simply call the *IressSessionEnd* method and set the **IRESSSessionKey** in the header.

User Scenarios

If the *IRESSSessionStart* method is successful, you will get the following:

- **IRESSSessionKey**: The token you use to call Iress Market Data methods

In the scenario where there are no more licenses available and you need to ensure you obtain a new session, you can either:

- End an existing session using *IRESSSessionEnd* (see [Session Management](#))
- End an existing session during the call to *IRESSSessionStart* in these two possible scenarios, List out the sessions in use, or Force close any existing sessions.

Scenario 1: List out the sessions in use

There will be some scenarios where all your available sessions (determined by your licence limit) are in use. When this happens, an exception will be returned with all the session details. You may already have a Viewpoint session, a Pro session and a web services session active as demonstrated below:

```
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd-
d="http://www.w3.org/2001/XMLSchema" xmlns:soap-
p="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Receiver</faultcode>
      <faultstring>Error: Login failed. No more licenses available for this login. Please con-
tact Support..</faultstring>
      <detail>
        <IRESSFaultDetail xmlns="http://webservices.iress.com.au/v4/" xmlns:i-
i="http://www.w3.org/2001/XMLSchema-instance">
          <Message>Error: Login failed. No more licenses available for this login. Please con-
tact Support..</Message>
          <Number>25008</Number>
          <Context><![CDATA[<CurrentSessions xmlns:x-
si="http://www.w3.org/2001/XMLSchema-instance">
            <Session><SessionNumber>1041878485</SessionNumber>
<LoginType>NetIressPro</LoginType><LoginDateTime>2020-09-23T21:55:54</Lo-
ginDateTime><LoginDuration>00:08:17</LoginDuration><ConnectionDescription>IP =
203.19.128.30:55303, LanIP = 172.25.224.43:55</Con-
nectionDescription><PhysicalDescription>THIRUVIKRAM.MANTHU@AU02-TMANTH-
NB3</PhysicalDescription></Session><Session>
          <SessionNumber>1041878583</SessionNumber>
```

```

<LoginType>NetIressPro</LoginType><LoginDateTime>2020-09-23T21:56:25</Lo-
ginDateTime><LoginDuration>00:07:46</LoginDuration><ConnectionDescription>IP =
203.19.128.30:50353, LanIP = 172.25.224.43:50</Con-
nectionDescription><PhysicalDescription>THIRUVIKRAM.MANTHU@AU02-TMANTH-
NB3</PhysicalDescription></Session><Session>
  <SessionNumber>1041878998</SessionNumber><LoginType>ViewPoint</Lo-
ginType><LoginDateTime>2020-09-23T21:58:48</Lo-
ginDateTime><LoginDuration>00:05:23</LoginDuration><ConnectionDescription>IP =
59.102.98.169</ConnectionDescription><PhysicalDescription>Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit</PhysicalDescription></Session></CurrentSessions>]]></Context>
  <Service>IRESS</Service>
  <Server>AU1-T-MASTERA:4501 (PHOENIX)</Server>
  <Method/>
  <UserName>buyside</UserName>
  <CompanyName>Iress</CompanyName>
  <EndPoint>https://webservices.iress.com.au/v4/SOAP.aspx</EndPoint>
  <RequestID>Login/Logout - E1B41264-6E13-4E47-A325-45ABDF502DF5</RequestID>
  <SessionKey/>
  <ServiceSessionKey/>
  <WebServiceTimeStamp>2020-09-23T22:04:12</WebServiceTimeStamp>
  <WebServiceServer>AU1-T-WEBSRVRA:46301</WebServiceServer>
  <WebServiceConnection>AU1-T-MASTERA:4501 (PHOENIX)</We-
bServiceConnection>
  <v4:Input xmlns:v4="http://webservices.iress.com.au/v4/">

```

A list of sessions and their numbers is returned in the Context field of the **IRESSFaultDetail** object when an *IRESSSessionStart* call fails because the user is out of licenses (Error number 25008).

List elements

The elements for the list are:

Element Name	Description	Type
CurrentSessions	The outermost element that contains all the sessions.	List of sessions
Session	The container for all the elements of a single session.	List of session elements

Session elements

The elements within each session are:

Element Name	Description	Type
SessionNumber	A unique identifier for the session. Pass this number back in the SessionNumberToKick parameter.	int
LoginType	A description of the type of login. This is usually the name of the Iress product but can be a general description like Web Services.	string
LoginDateType	The date and time that the session began.	dateTime
LoginDuration	The time that the session has been running.	time
ConnectionDescription	A description of the connection.	string
PhysicalDescription	A description of the session.	string



Note: More elements may be added to this table in future.

You can then call *IRESSSessionStart* again, this time setting the **SessionNumberToKick** parameter to the session number that you want to end.

We recommend setting **KickLikeSessions=True** in this scenario so that, if a nominated session has already ended, a “like” session of the same login type as the nominated session will be ended.



WARNING: You may still not be able to start a new session if there are no licenses available and new sessions have started - which are of a different type to the session provided in **SessionNumberToKick** and **KickLikeSessions=True** - since the previous *IressSessionStart* call.

Scenario 2: Force close any existing session

If you reach your license limit and need to end any existing session that is in use, the **SessionsNumberToKick** parameter can be passed a value of -1. This will end all existing sessions.



Tip: This scenario greatly simplifies session management.

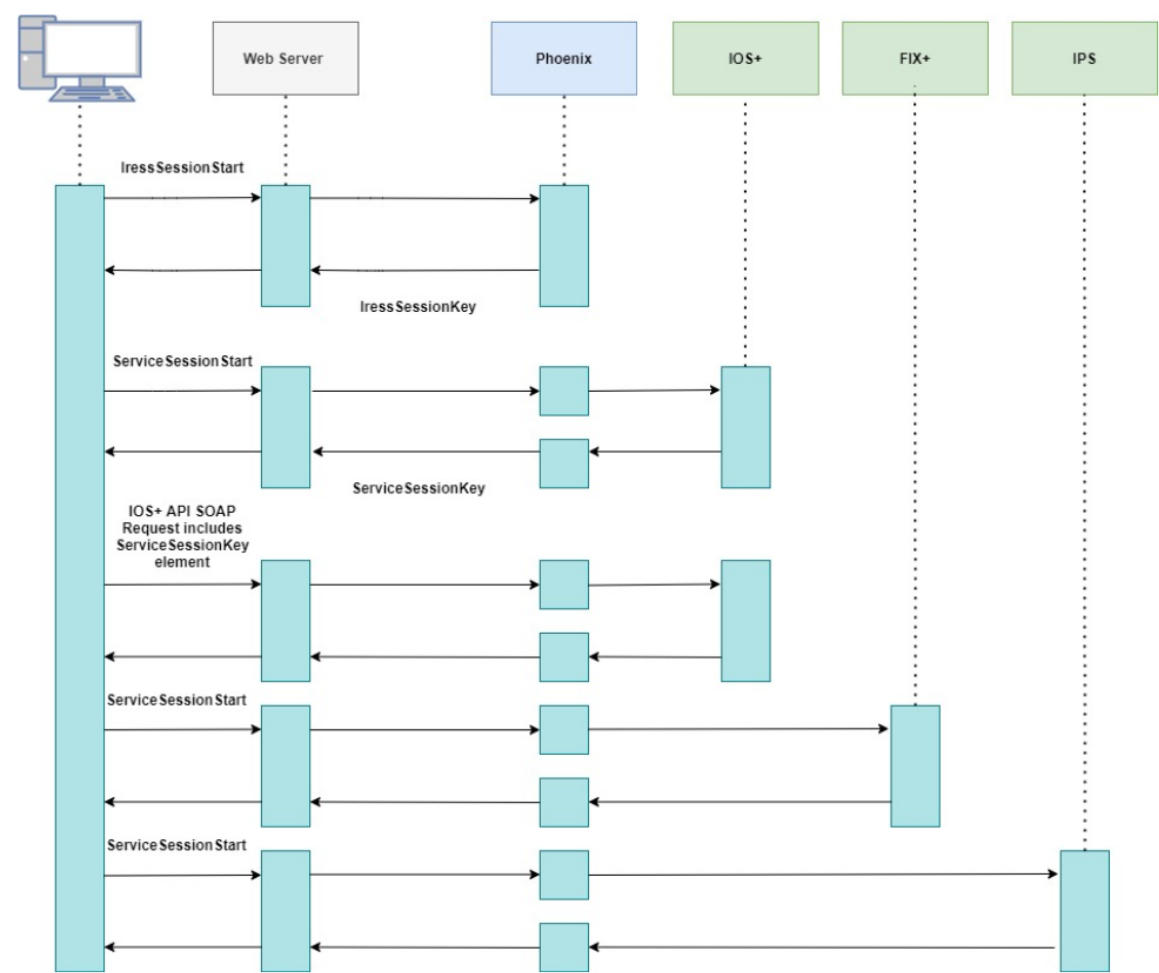
• **SessionNumberToKick** as -1

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:v4-4="http://webservices.iress.com.au/v4/">
  <soapenv:Header/>
  <soapenv:Body>
    <v4:IRESSSessionStart>
      <v4:Input>
        <v4:Header>
          <v4:SessionKey></v4:SessionKey>
          <v4:RequestID></v4:RequestID>
        </v4:Header>
      </v4:Input>
    </v4:IRESSSessionStart>
  </soapenv:Body>
</soapenv:Envelope>
```

```

    <v4:Updates>>false</v4:Updates>
    <v4:Timeout>25</v4:Timeout>
    <v4:PageSize>0</v4:PageSize>
    <v4:WaitForResponse>true</v4:WaitForResponse>
    <v4:PagingBookmark>
      <!--You may enter ANY elements at this point-->
    </v4:PagingBookmark>
    <v4:PagingDirection>0</v4:PagingDirection>
    <!--Optional:-->
    <v4:InputLocalizationType>0</v4:InputLocalizationType>
    <!--Optional:-->
    <v4:OutputLocalizationType>0</v4:OutputLocalizationType>
    <!--You may enter ANY elements at this point-->
  </v4:Header>
  <v4:Parameters>
    <v4:UserName>username</v4:UserName>
    <v4:CompanyName>Company</v4:CompanyName>
    <v4>Password>Password</v4>Password>
    <v4:ApplicationID>AppID</v4:ApplicationID>
    <v4:ApplicationLabel>AppLabel</v4:ApplicationLabel>
    <v4:PreviousSessionKey></v4:PreviousSessionKey>
    <v4:SessionTimeout></v4:SessionTimeout>
    <v4:AuthenticationType></v4:AuthenticationType>
    <v4:SessionNumberToKick>-1</v4:SessionNumberToKick>
    <v4:KickLikeSessions>true</v4:KickLikeSessions>
    <v4:Locale></v4:Locale>
    <v4:LocalePrivateUseSubtags></v4:LocalePrivateUseSubtags>
  </v4:Parameters>
  <!--You may enter ANY elements at this point-->
</v4:Input>
</v4:IRESSSessionStart>
</soapenv:Body>
</soapenv:Envelope>

```



Session Error Management & Session Recovery

 Go [here](#) for application error management.

Session Error Management

Note that an Iress session is always a parent and service sessions are always children of an Iress session.

Example:

1. A user creates an Iress session by calling *IRESSSessionStart* with their username, password, etc.
2. The server returns an **IRESSSessionKey** to the user.
3. The user can use the **IRESSSessionKey** to call APIs against the Iress session such as:
 - *PricingQuoteExGet*
 - *PricingTradeHistoricalGet*
4. The user wants to create a session against an IOS+ so they can place orders and trade. They call *ServiceSessionStart* with the **IRESSSessionKey** obtained in #2
5. The IOS+ server returns a **ServiceSessionKey** to the user.
6. The user can use the **ServiceSessionKey** to call APIs against the IOS+ service session such as:
 - *OrderPadGetByUser*
 - *OrderCreate*

Error handling and mitigation

When a service session ends (regardless of whether it was invoked by the user or invoked by the server), the **ServiceSessionKey** can no longer be used. A user may end a service session by calling *ServiceSessionEnd* or *IRESSSessionEnd*. A server may end

the service session for various reasons (e.g. permission changes, exceeding the session time limit, exceeding the idle time limit, etc.).

If the **ServiceSessionKey** is used after a service session is ended, an error will be returned. These are the possible error codes:

- 25006
- 25009
- 25019
- 25033

Refer to the error message in the response for a brief description of the error. When this occurs, a new service session needs to be established.

It may also be necessary to establish a new Iress session - in the case where the Iress session is ended (and therefore all its child service sessions are also ended). If the attempt to establish a new service session (by calling *ServiceSessionStart*) fails with error codes:

- 25014
- 25019
- 25022

This would indicate that the parent Iress session has ended and a new Iress session needs to be established.

Best-practice workflow

1. Create an Iress session via *IRESSSessionStart* with unique **ApplicationID**
2. Create a service session (e.g. to an IOS+ server) via *ServiceSessionStart*
3. Make requests against the service session using the **ServiceSessionKey** obtained from step #2

Either,

1. The service session ends (e.g. invoked by the server or invoked by the client)
 - Subsequent requests using the **ServiceSessionKey** will fail with error codes 25006, 25009, 25019 or 25033
 - Go to step #2

Or,

1. The Iress session ends (e.g. invoked by the server or invoked by the client)

- Subsequent requests using the **IRESSSessionKey** will fail with error codes 25014, 25019 or 25022
- Go to step #1

 You can see the list of possible session related error codes under [Appendix](#). This list is not exhaustive and does not cover Application errors.

Recovery

From within a given thread, if the client does not receive an **IRESSSessionKey** from the *IRESSSessionStart* method due to a timeout, log in again with the same credentials (**UserName** / **Company** / **ApplicationID** / **ApplicationLabel**). If an Iress session was created in the backend with those credentials, it will be returned. If it wasn't, a new Iress session will be created and an **IRESSSessionKey** returned.

If a client application fails, and you need to restart and login again, you should call *IRESSSessionStart* with a new application ID to get a new session. If you receive a session limit error, follow the steps detailed in [User Scenarios](#)

For service sessions, if the client does not receive the service session key from the *ServiceSessionStart* method due to a timeout, simply call *ServiceSessionStart* again.

Alternatively, you can manually manage your session from the Iress front end:

1. Choose **Commands > Admin > User Manager**
2. Select the user you want to manage
3. Right-click then select **Logout / Disconnect**

Should the client be disconnected from the server (e.g. due to a network disruption, and not able to reconnect to close the session to release the login etc.). It is not necessary to "reconnect" to close the session. Once the network disruption is over, the client can call *IRESSSessionEnd* with the **IRESSSessionKey** from their previous session to end that session.

There are also options to kick an existing session to free the login license for the new session – refer to [User Scenarios](#).

Session Expiration

Session expiration is set at User group level and company level in Web Administration. The default setting is 1:00 AM in the server time zone. This is configurable and can be set to either a fixed time, or a duration.

The **SessionTimeout** parameter can be passed in to *IRESSSessionStart* to override this setting. The maximum number of minutes supported by this parameter is 1440 minutes (24 hours).



Note: This session timeout is not precise and may happen up to 1 minute after the specified value.

In addition to this, there is an idle session timeout of 2 hours. If there is no activity on an Iress session for a 2 hour period then the session will be expired by the server automatically.

Paging

In this topic:

- [Paging Overview](#)
- [Paging with bookmarks](#)
- [Paging in IPS](#)

Overview

Web Services V4 supports the retrieval of data in pages to support the breaking up of large responses into smaller pieces. You can specify the maximum number of rows of data each page can contain by setting the **PageSize** property on the input header of the request.

The server will return a response containing a row-count equal to the minimum of the value provided in **PageSize**, and the *maximum* allowed rows per page for the given method being called. Some methods may have smaller pages as they return a large number of columns.

When a response to a method call indicates there is more data pending (status code = 1), the method needs to be called again on the same **RequestID** to get the next page of data; this should be done until all of the data has returned (status code of 2).

A small number of methods support paging using bookmarks; these allow a client to request data from a particular point, for example resuming data retrieval from the last known record instead of having to start all over again in a recovery scenario.



Tip: Avoid writing code which assumes a response may contain a fixed number of rows as the maximum number of rows per page may be tweaked on any given method.

Paging with bookmarks

Some methods support paging using bookmarks. This allows specifying the effective starting point to retrieve data from when a method is called. The pattern for paging described above, including calling on the same **RequestID** to get the next page of data still applies.

Methods that support bookmarks can be identified in the method reference; they show a bookmark icon next to the output fields making up the bookmark.

A sample is shown below:

 AuditTrailGetByAccount**Description**

Retrieve audit trail information using an account filter.

SyntaxAuditTrailGetByAccountOutput *Output* = [object].**AuditTrailGetByAccount** (AuditTrailGetByAccountInput *Input*)**Properties**

Deprecated	No
Supports Updates	No

Input Parameters

Pos	Name	Type	Size*	Nullable?	Default Value	Array?	Array Size	Description
1	AuditLogDateTimeFrom	dateTime	8	Yes		No		Start date and time for retrieving audit trail data. (Server time).
2	AuditLogDateTimeTo	dateTime	8	Yes		No		End date and time for retrieving audit trail data. (Server time).
3	EventUserCode	string	100	Yes		No		User code of the person causing the audit trail event.
4	AccountCode	string	61	No		Yes	1	The account code to filter by.
5	SecurityCode	string	32	Yes		No		The security code to filter by.
6	Exchange	string	16	Yes		No		The exchange to filter by.
7	OrderFlagsMask	int64	8	Yes		No		A bitmask containing various boolean properties of the order. The OrderFlagG
8	OrderFlagsMask2	int64	8	Yes		No		Second 64 bits bitmask containing various boolean properties of the order. Tf

Output Columns

Pos	Name	Type	Size*	Nullable?	Default Value	Description
1	AuditTrailNumber	int64	8	No		Unique audit trail identifier.
2	OrderNumber	int64	8	No		Unique IOS order number.
3	ParentOrderNumber	int64	8	Yes		Unique IOS parent order number. Populated with the OrderNumber from the
4	AuditLogDateTime	dateTime	8	Yes		Date and time when the audit trail entry was logged. (Server time).
5	ExchangeDateTime	string	23	Yes		Date and time the order was updated at the exchange.
6	AccountCode	string	61	Yes		Name of the account to which the order has been allocated.
7	SecurityCode	string	32	Yes		Exchange code of the security.
8	Exchange	string	16	Yes		Exchange on which the security is traded.
9	Destination	string	41	Yes		Destination of the order.

```

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:v4-
4="http://webservices.iress.com.au/v4/">
  <soapenv:Header/>
  <soapenv:Body>
    <v4:AuditTrailGetByAccount>
      <v4:Input>
        <v4:Header>
          <v4:ServiceSessionKey>22A4C1EE-48DE-477D-A02F-3E9D2CE39D77@We-
bServicesTestA.iress.com.au</v4:ServiceSessionKey>
          <v4:RequestID>test123</v4:RequestID>
          <v4:Updates>>false</v4:Updates>
          <v4:Timeout>25</v4:Timeout>
          <v4:PageSize>0</v4:PageSize>
          <v4:WaitForResponse>true</v4:WaitForResponse>
          <v4:PagingBookmark>
            <v4:AuditTrailNum-
ber>LastAudittrailNumberFromPreviousResponse</v4:AuditTrailNumber>
            <!--You may enter ANY elements at this point-->
          </v4:PagingBookmark>
          <v4:PagingDirection>0</v4:PagingDirection>
          <!--Optional:-->
          <v4:InputLocalizationType>0</v4:InputLocalizationType>
          <!--Optional:-->
          <v4:OutputLocalizationType>0</v4:OutputLocalizationType>
          <!--You may enter ANY elements at this point-->
        </v4:Header>
        <v4:Parameters>
          <v4:AuditLogDateTimeFrom>2018-01-01</v4:AuditLogDateTimeFrom>

```

```

    <v4:AuditLogDateTimeTo>2020-09-24</v4:AuditLogDateTimeTo>
    <v4:AccountCodeArray>
      <!--Zero or more repetitions:-->
      <v4:AccountCode>AccCode</v4:AccountCode>
    </v4:AccountCodeArray>
  </v4:Parameters>
  <!--You may enter ANY elements at this point-->
</v4:Input>
</v4:AuditTrailGetByAccount>
</soapenv:Body>
</soapenv:Envelope>

```

Paging in IPS

There are some IPS methods in which the typical paging calling pattern does not apply. This is typically for older methods that were developed before Web Services V4 was first released, where the paging parameters were passed in the input parameters.

For example:

- *IPSAccountGetAll1*
- *IPSPositionGetAll1*

When calling such methods, a response may show the data is ready (status code = 2), yet there may still be more data waiting to be accessed on the IPS server. In this scenario, the method needs to be called again with a new **RequestID**, this time passing the last paging bookmark as the parameter for the new method call. This should be continued until there are no more records returned.

For example with *IPSAccountGetAll1*:

Request:

```

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:v4-
4="http://webservices.iress.com.au/v4/">
  <soapenv:Header/>
  <soapenv:Body>
    <v4:IPSAccountGetAll1>
      <v4:Input>
        <v4:Header>
          <v4:ServiceSessionKey>664b5291-5e45-4849-8d1c-74ca38402cd0@We-
bServicesTestA.iress.com.au</v4:ServiceSessionKey>
          <v4:RequestID>TESTPAGING</v4:RequestID>
          <v4:Updates>>false</v4:Updates>
          <v4:Timeout>25</v4:Timeout>
          <v4:PageSize>0</v4:PageSize>
          <v4:WaitForResponse>>true</v4:WaitForResponse>
          <v4:PagingBookmark>
            <!--You may enter ANY elements at this point-->
          </v4:PagingBookmark>
        </v4:Header>
      </v4:Input>
    </v4:IPSAccountGetAll1>
  </soapenv:Body>
</soapenv:Envelope>

```

```

    <v4:PagingDirection>0</v4:PagingDirection>
    <!--Optional:-->
    <v4:InputLocalizationType>0</v4:InputLocalizationType>
    <!--Optional:-->
    <v4:OutputLocalizationType>0</v4:OutputLocalizationType>
    <!--You may enter ANY elements at this point-->
  </v4:Header>
  <v4:Parameters>
    <v4:PageSize>200</v4:PageSize>
    <v4:PreviousAccountCode>?</v4:PreviousAccountCode>
  </v4:Parameters>
  <!--You may enter ANY elements at this point-->
</v4:Input>
</v4:IPSAccountGetAll1>
</soapenv:Body>
</soapenv:Envelope>

```

Response:

```

<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd=
d="http://www.w3.org/2001/XMLSchema" xmlns:soap=
p="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <IPSAccountGetAll1Response xmlns="http://webservices.iress.com.au/v4/">
      <Output>
        <v4:Input xmlns:v4="http://webservices.iress.com.au/v4/">
          <v4:Header>
            <v4:ServiceSessionKey>664b5291-5e45-4849-8d1c-74ca38402cd0@We-
bServicesTestA.iress.com.au</v4:ServiceSessionKey>
            <v4:RequestID>TESTPAGING</v4:RequestID>
            <v4:Updates>>false</v4:Updates>
            <v4:Timeout>25</v4:Timeout>
            <v4:PageSize>0</v4:PageSize>
            <v4:WaitForResponse>>true</v4:WaitForResponse>
            <v4:PagingBookmark>
              <!--You may enter ANY elements at this point-->
            </v4:PagingBookmark>
            <v4:PagingDirection>0</v4:PagingDirection>
            <!--Optional:-->
            <v4:InputLocalizationType>0</v4:InputLocalizationType>
            <!--Optional:-->
            <v4:OutputLocalizationType>0</v4:OutputLocalizationType>
            <!--You may enter ANY elements at this point-->
          </v4:Header>
          <v4:Parameters>
            <v4:PageSize>200</v4:PageSize>
            <v4:PreviousAccountCode>?</v4:PreviousAccountCode>
          </v4:Parameters>

```

```

    <!--You may enter ANY elements at this point-->
  </v4:Input>
  <Result>
    <Header>
      <RequestID>TESTPAGING</RequestID>
      <StatusCode>2</StatusCode>
      <WebServiceTimeStamp>2020-09-28T19:51:45</WebServiceTimeStamp>
      <PagingBookmark/>
    </Header>
    <HeaderRow/>
    <DataRows>
      .
      .
      .
    <DataRow>
      <AccountCode>AccountCode</AccountCode>
      <AccountName>Asia Market Neutral</AccountName>
      <AccountName2/>
      <AccountName3/>
      <ActiveDate xsi:nil="true"/>
      <AdvisorCode/>
      <AllocationTargetName/>
      <BaseCurrency>AUD</BaseCurrency>
      <CorporateActionAccrualExchange/>
      <CorporateActionAccrualSecurityCode/>
      <DefaultCashExchange>UNL</DefaultCashExchange>
    </DataRow>
  </DataRows>

```

The last account returned in the previous response should be recorded, then passed in as the **PreviousAccountCode** for the new method call (using a new **RequestID**):

```

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:v4-
4="http://webservices.iress.com.au/v4/">
  <soapenv:Header/>
  <soapenv:Body>
    <v4:IPSAccountGetAll1>
      <v4:Input>
        <v4:Header>
          <v4:ServiceSessionKey>817ab46b-f588-4bc8-beb1-0286d7f516ff@We-
bServicesTestA.iress.com.au</v4:ServiceSessionKey>
          <v4:RequestID>TESTPAGING1</v4:RequestID>
          <v4:Updates>false</v4:Updates>
          <v4:Timeout>25</v4:Timeout>
          <v4:PageSize>0</v4:PageSize>
          <v4:WaitForResponse>true</v4:WaitForResponse>
          <v4:PagingBookmark>
            <!--You may enter ANY elements at this point-->
          </v4:PagingBookmark>
          <v4:PagingDirection>0</v4:PagingDirection>
        </v4:Header>
      </v4:Input>
    </v4:IPSAccountGetAll1>
  </soapenv:Body>
</soapenv:Envelope>

```

```
<!--Optional:-->
<v4:InputLocalizationType>0</v4:InputLocalizationType>
<!--Optional:-->
<v4:OutputLocalizationType>0</v4:OutputLocalizationType>
<!--You may enter ANY elements at this point-->
</v4:Header>
<v4:Parameters>
  <v4:PageSize>200</v4:PageSize>
  <v4:PreviousAccountCode>PreviousAccountCode</v4:PreviousAccountCode>
</v4:Parameters>
<!--You may enter ANY elements at this point-->
</v4:Input>
</v4:IPSAccountGetAll1>
</soapenv:Body>
</soapenv:Envelope>
```

Updates

Some methods support the ability to query live updates to data. This is facilitated using a long polling mechanism.

There are two ways to subscribe to updates:

- Retrieving historical data and watching for updates on that data, e.g. retrieving existing orders in order pad and subscribing to any updates on them
- Retrieving updates on their own, e.g. retrieving trade updates in order to produce a trade ticker

By convention, the naming of an updates method will have “Updates” on the end of the method name. For example, *PricingQuoteGetUpdates*.

To identify whether a method supports updates, refer to the method reference and look out for “Supports Updates” and “Updates Method”:

PricingQuoteGet

Description

Retrieves basic quote information for one or more securities.
A security code must be specified in the *SecurityCode* or *SecurityText* parameter.
If both *SecurityCode* and *SecurityText* are given, *SecurityCode* is used.

Syntax

PricingQuoteGetOutput Output = [object].**PricingQuoteGet** (*PricingQuoteGetInput Input*)

Properties

Deprecated	No
Supports Updates	Yes
Supports Localization	No
Updates Only	No
Updates Method	PricingQuoteGetUpdates

To subscribe to updates, set the **Updates** field in the input header to True.

The response from the server will either have **StatusCode** 1 (indicating there are more pages to retrieve) or 3 (indicating the backend is now watching for updates).

A request will keep watching for updates until one of the following happens:

- The request expires, which happens after 30 minutes of inactivity
- The queued updates exceed the updates queue limit of 10,000 updates (this limit

applies per request)

- The client terminates the request (i.e. using *SessionRequestEnd*)
- The client terminates the session

In order to request historical data and subscribe to updates at the same time:

- Call the data request with **Updates=True**
- Retrieve all pages until **StatusCode = 3**
- Continuously call the updates method with the same **RequestID**



Note: When there are no updates available the backend waits for new data for up to one second before sending a response.



Note: It is quite common for data requests to contain a more extensive set of filters than what is supported on the updates request. Where you are asking for updates on a data request, you can identify which fields are applied to updates simply by looking at the input parameters on the matching Updates method. Any additional filtering required on updates should be performed by the calling application.



Note: In order to prevent hitting the limit on 10,000 maximum queued updates for a given request, either long poll more frequently without overlap (i.e. wait for a response to arrive before attempting a new call), or if this doesn't mitigate the problem then break up the request into multiple requests by using additional filtering.

Synchronous vs Asynchronous Calling

By default, method calls are synchronous. Which means the method call will block until the response data is available and sent from the server. This is controlled by the **WaitForResponse** input header parameter, which defaults to True.

To make an asynchronous method call, set **WaitForResponse** to False. When an asynchronous method call is received by the server, it will return immediately with **StatusCode** set to 1 (indicating there are more pages to be retrieved). Call back in again with the same **RequestID**; if the response data is not yet available, the server will again return immediately with **StatusCode** set to 1. Repeatedly call back in with the same **RequestID** until the server responds with the data.

API Methods by Application

Methods by Product

- [Market Data and Web Admin](#)
- [IOS+](#)
 - [XML fields in <MiscFees>](#)
 - [Contingent Orders](#)
 - [Algo Orders](#)
- [IPS](#)
- [FIX+](#)

All the Web Services V4 samples linked in this document are available at this public GitHub repo: <https://github.com/iress/web-services-v4-docs>.



The samples include examples of SOAP XML requests and responses for key methods across each application, as well as a handful in different programming languages. To open a sample, click on the **Request** or **Response** column under the method you want to use.

Iress Pro

All the Web Services V4 samples linked in this document are available at this public GitHub repo: <https://github.com/iress/webservices-v4-docs>.



The samples include examples of SOAP XML requests and responses for key methods across each application, as well as a handful in different programming languages. To open a sample, click on the **Request** or **Response** column under the method you want to use.

Market Data and Web Admin

This topic shows some sample SOAP XML requests and responses for market data methods. All market data and web admin methods can be accessed with just an Iress session key. These samples can be used in conjunction with method reference documentation to pass in relevant parameters.

IRESSSessionStart	
Request	Response
TimeSeriesGet2	
Request	Response
PricingQuoteGet	
Request	Response

IOS+

Go [here](#) for information on [contingent orders](#) (CO)

Go [here](#) for information on [algo orders](#)

All the Web Services V4 samples linked in this document are available at this public GitHub repo: <https://github.com/iress/web-services-v4-docs>.



The samples include examples of SOAP XML requests and responses for key methods across each application, as well as a handful in different programming languages. To open a sample, click on the **Request** or **Response** column under the method you want to use.

This section shows some sample SOAP XML requests and responses for IOS+ methods.

ServiceSessionStart for IOS+ service	
Request	Response
Create order (OrderCreate3)	
Request	Response
Amend order (OrderAmend2)	
Simple volume amend.	
Request	Response
Cancel order (OrderDelete)	
Request	Response
OrderPadGetByAccount and OrderPadGetByAccountUpdates	
OrderPadGetByAccount or other OrderPadGetBy* methods are used to get a snapshot of the order pad and can be used in conjunction with their own updates method to receive ongoing updates for the orders received or new orders as well. If you are intending to watch for updates please set the updates parameter to true for both methods	
OrderPadGetByAccount: Request	OrderPadGetByAccount: Response
OrderPadGetByAccountUpdates: Request	OrderPadGetByAccountUpdates: Response

You need to pass the same **RequestID** for updates method which you used for *OrderPadGetByAccount* method.

Get bookings (BookingGetByOrganisation2)

Request

Response

Explanation of XML fields in <MiscFees> tags

Set: Whether this fee has been set on the booking;

1 - fee set for booking

FeeType: Type of the fees, following list shows the meaning of the number return by this field

- **Unknown** = 0, /*Used for internal (Iress) purposes only*/
- **Regulatory** = 1,
- **GST** = 2, /*Named 'Tax' in Fix Protocol (<http://fix-wiki.fixprotocol.org/fixwiki/MiscFeeType>)*/*
- **LocalCommission** = 3,
- **ExchangeFees** = 4,
- **Stamp** = 5,
- **Levy** = 6,
- **Other** = 7,
- **Markup** = 8,
- **ConsumptionTax** = 9,
- **PerTransaction** = 10,
- **Conversion** = 11,
- **Agent** = 12,
- **TransferFee** = 13,
- **SecurityLending** = 14,
- **Research** = 15

FeeBasis: Used with **Properties** to calculate **FeeAmount**

- **Absolute** = 0
- **PerUnit** = 1
- **Percentage** = 2

Properties: used with **FeeBasis** to calculate **FeeAmount**, only relevant if **FeeBasis** = 2 (Percentage)

1. Percentage of commission value = 0
2. Percentage of order value = 1

FeeAmount: calculated fee amount for this fee type, e.g.: if fee rate = 10 and **FeeBasis** = 0 (Absolute), then **FeeAmount** = 10

1. if fee rate = 10 and **FeeBasis** = 1 (PerUnit), then **FeeAmount** = 10 * Volume of the booking
2. if fee rate = 10 and **FeeBasis** = 2 (Percentage) and **Properties** = 0, then **FeeAmount** = 10% * booking commission value
3. if fee rate = 10 and **FeeBasis** = 2 (Percentage) and **Properties** = 1, then **FeeAmount** = 10% * booking value

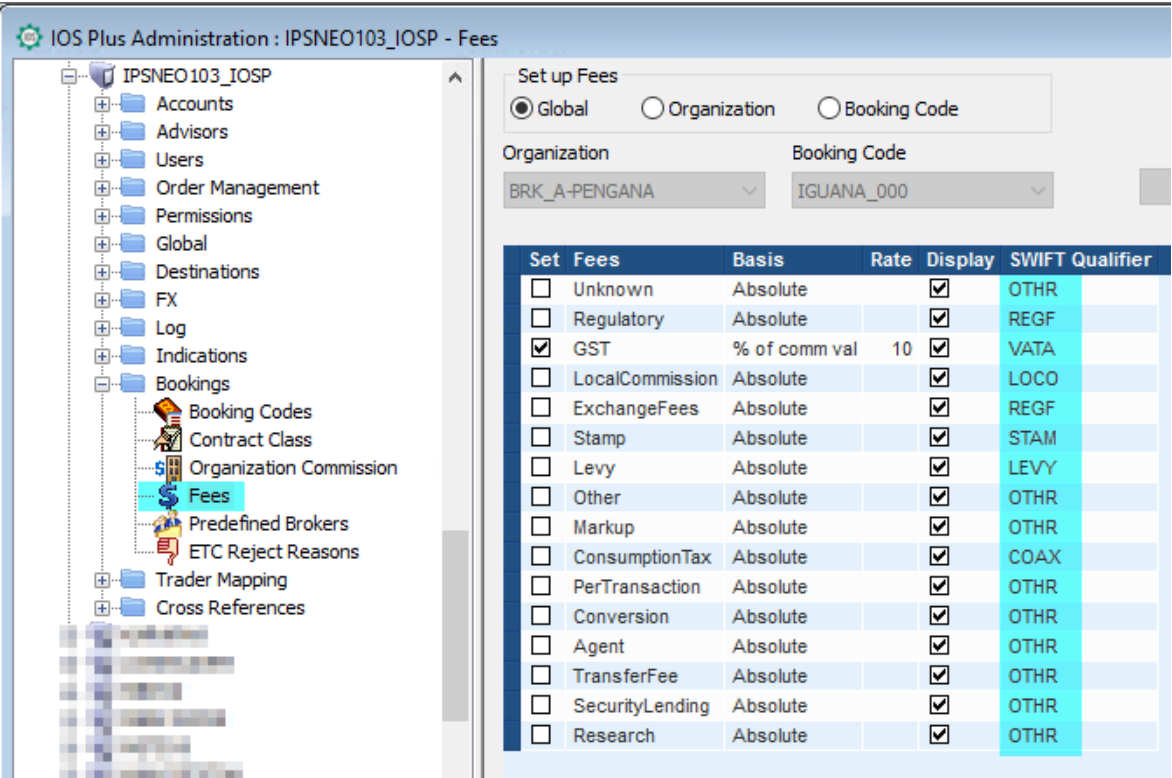
Currency: used by client allocation only

Display: whether this field will be displayed in IOS+ booking command (not quite relevant to API user), can be set from IOS+ Administration > Bookings > Fees:

SWIFTQualifier: SWIFT qualifier for corresponding fee type, the value is stored in market interface DB, here is list of examples in production:

IOSFeeType	SWIFTQualifier
15	OTHR
14	OTHR
13	OTHR
12	OTHR
11	OTHR
10	OTHR
9	COAX
8	OTHR
7	OTHR
6	LEVY
5	STAM
4	REGF
3	LOCO
2	VATA
1	REGF
0	OTHR

You can also see the SWIFTQualifier for each type in **IOS+ Administration > Bookings > Fees:**



Get ETCs (ETCGetByOrganisation)	
<u>Request</u>	<u>Response</u>

IOS+ Contingent Orders

Contingent Orders are built on top of our standard order entry and can be accessed using methods in the Order category. For example, to create/amend/cancel contingent orders you would use `OrderCreate3`, `OrderAmend2`, and `OrderDelete` respectively. To retrieve contingent orders, you can use any of the `OrderPadGetBy*` or `OrderSearchGetBy*` methods.

Contingent order destinations

There are four different contingent order destinations available in IOS+:

- [FIXED CO](#)
- [TRAILING CO](#)
- [OCO](#)
- [IFDONE](#)

Contingent order destinations can be identified using a combination of the **DestinationGet** and **DestinationDetailGet** methods.

To identify contingent order destinations:

1. Call the *DestinationGet* method with **AccessMode** = 0, destinations you have access to will be returned
2. Pass the retrieved destinations into the *DestinationDetailGet* method, the details for each destination will be returned
3. Contingent order destinations can be identified by a **DestinationSubTypeNumber** between 30 and 49.



Note: The *DestinationDetailGet* method needs the "view destination" permission enabled. If this permission cannot be enabled, we recommend maintaining a list of available contingent order destinations in the calling application

Order attributes – Iress strategy properties

Each contingent order destination can have different order attributes available that can be set on order entry. These attributes would typically be available under Iress Strategy Properties – in the screenshot below this is the grid in the bottom left corner.

One option you can use to determine what attributes are relevant for a contingent order (CO) against a CO destination is to:

1. Create an order in an Iress front-end, such as Pro or ViewPoint
2. Call one of the retrieval methods, such as *OrderPadGetByAccount*
3. Observe what is set in **Execution Instructions** in IOS+ Order Entry

The **Execution Instructions** will be populated with the CO's attributes.

Another option is to use the *AttributeGetByUser* method to retrieve a list of attributes available for order entry. For example, to filter out the Iress Strategy Properties specifically, the **AttributeCategoryNumber** input parameter on this method should be set to 8 to retrieve these specific attributes.

The following is a table with examples of supported contingent order destinations and the attributes that are relevant to each, although may be extended over time and not complete:

FIXED CO

Attribute	Description	Type	List values
IS013	Contingency security	Security.Exchange	

Attribute	Description	Type	List values
IS016	Contingent order status	String (set by STI)	Active, Triggered, Deleted, Pre-Active
IS020	Contingency base price	String	Last, Bid, Ask
IS021	Condition	String	Greater, Less, Equal, Greater or Equal, Less or Equal
IS022	Price	Float	
IS028	Email on trigger	Bool	
IS029	Email on partial trade	Bool	
IS030	Email on full trade	Bool	
IS031	Emailed on failed	Bool	
IS032	Email on expiry	Bool	
IS034	Email on untraded order	Bool	
IS035	Audit trail trigger event	Bool	
IS036	Audit trail partial trade	Bool	
IS037	Audit trail full trade	Bool	
IS038	Audit trail failed	Bool	
IS039	Audit trail expiry	Bool	
IS040	Audit trail untraded order	Bool	
IS041	Email address	String	

TRAILING CO

Attribute	Description	Type	List values
IS013	Contingency security	Security.Exchange	
IS014	Contingency base price	String	Last
IS016	Contingent order status	String (set by STI)	Active, Triggered, Deleted, Pre-Active
IS025	Trailing type	String	Price, Percentage

Attribute	Description	Type	List values
IS026	Trailing direction	String	Above the base price, Below the base price
IS027	Trailing offset	Float	
IS028	Email on trigger	Bool	
IS029	Email on partial trade	Bool	
IS030	Email on full trade	Bool	
IS031	Emailed on failed	Bool	
IS032	Email on expiry	Bool	
IS034	Email on untraded order	Bool	
IS035	Audit trail trigger event	Bool	
IS036	Audit trail partial trade	Bool	
IS037	Audit trail full trade	Bool	
IS038	Audit trail failed	Bool	
IS039	Audit trail expiry	Bool	
IS040	Audit trail untraded order	Bool	
IS041	Email address	String	

OCO

Attribute	Description	Type	List values
IS016	Contingent order status	String (set by STI)	Active, Triggered, Deleted, Pre-Active

IFDONE

Attribute	Description	Type	List values
IS016	Contingent order status	String (set by STI)	Active, Triggered, Deleted, Pre-Active

Order placement with take profit

This example uses the combinations of the IFDONE, AUTODESK and FIXED CO destinations.

The main order is created on the AUTODESK destination. The order on the FIXED CO destination will only become active and ready to trigger if the order on the AUTODESK destination is either partially or fully traded.

If the conditions for the take profit are met, the take profit order on the FIXED CO destination will trigger.

Order placement with take profit	
Request	Response

IOS+ External Algo Orders

Similar to [contingent orders](#), external algo orders are built on top of our standard order entry. Hence they can be accessed using methods in the Order category. For example, to create/amend/cancel algo orders, you would use the *OrderCreate3*, *OrderAmend2* and *OrderDelete* methods respectively. To retrieve external algo orders, you would use any of the *OrderPadGetBy** or *OrderSearchGetBy** methods.

Algo destinations

In IOS+ Administration, an algo destination can be identified by viewing the detail against each destination and observing whether the destination has a **Destination type** of "EXTERNAL ALGO" and where the destination property **Algo** is enabled:

Destination Detail - ALGO TEST

Destination Detail | Exchanges | Order Attributes | Extended Properties

Destination name: ALGO TEST

Parent destination: FIXTI

Default account: UNKNOWN

Destination type: EXTERNAL ALGO

Booking destination:

Target destination:

Destination routing: Routed

Translation group:

Show Group

Destination description:

☒ Requires server interface

☒ Orders at risk

☒ Amendable orders

☐ Automatically increase order volume when grouping new orders

☐ Create grouping parent orders with zero volume and price for externally created orders

Destination properties: Enter filter text

Enable ▾ Property

- ☒ Orders processed as at-risk
- ☒ Algo
- ☐ Done For Day read only
- ☐ Do not update destination booking codes in synchronisation

Algo FIX properties

Tag: Value:

FIX properties

Broker Code:

OK Cancel Help

You can identify external algo destinations by using a combination of the *DestinationGet* and *DestinationDetailGet* methods.

To identify external algo destinations:

1. Call the *DestinationGet* method with **AccessMode** = 0 , destinations you have access to will be returned

2. Pass the retrieved destinations into the *DestinationDetailGet* method, the details for each destination will be returned
3. You can then identify external algo destinations by matching the **DestinationType** output column to "EXTERNAL ALGO"

You can perform an additional check to confirm if **Algo** destination property is enabled by confirming the **DestinationPropertiesMask** output column contains the **Algo** bit (256).



Note: The *DestinationDetailGet* method needs the "View destination" permission enabled. If this permission cannot be enabled, we recommend maintaining a list of available contingent order destinations in the calling application.

Order attributes – External algo properties

External algo properties are set against an order via execution instructions. This is achieved via either the **ExecutionInstructions** or **ExecutionInstructionsDictionary** parameter in the *OrderCreate3/OrderAmend2* methods in IOS+; or when querying the details against an order via the *OrderPadGetBy** or *OrderSearchGetBy** methods (as an output column).

One option you can use to determine what attributes are relevant for an external algo order against an algo destination is to:

1. Create an order in the Pro front-end
2. Call one of the retrieval methods, such as *OrderPadGetByAccount*
3. Observe what is set in **Execution Instructions** in IOS+ Order Entry

The **Execution Instructions** will be populated with the external algo attributes.

IOS Plus Algo Order Entry: Create New Order

BUY SYM-BHP VOL-100 PRC-5050 End of Day

Buy: BHP Group Limited, IOS: 10, Security: BHP.ASX, Volume: 100, Price: 5001.00, C, 12:01:54 AM

Sell: AEDEMOP, BHP.ASX, 100, 5050, Limit, End of Day, XRef: VIKMATTACC

Short: IOS Account: ALGO TEST, ClrID: 2, OrderMatchID:

Algo Properties

Set	Description	Value
☒	Algo Properties	
☒	OrderVol	
☒	Wire Test	1
☒	Urgency	Medium
☒	starttime	
☒	Strategy Name	ALGO TEST
☒	I Would Price	
☒	Execution Instructions	

Another option is to use the *AttributeGetByUser* method to retrieve a list of attributes available for order entry. For example, to filter out the External Algo Properties specifically, the **AttributeCategoryNumber** input parameter on this method should be set to 5 to retrieve these specific attributes.

Order attributes:

Enter filter text

Global Attributes	Description	DataType	ValueRange	DefaultValue	Disabled	Hidden	ReadO...	NotAmenda...	RestrictedToSide
☒	Pricing Instructions								
☒	Lifetime								
☒	Execution Instructions								
☒	Trade Markers								
☒	Custom Order Columns								
☒	External Algo Properties								
☒	110 Min Dark Fill Qty	Number							
☒	9043 Relative Limit Type	String Drop Down	<AP>Arriv...						
☒	9044 I Would Price	Float	I Would Pr...						
☒	9126 End Time	DateTime							
☒	9133 Max Dark Exposure	Number							
☒	9134 Nighthawk Limit Px	Float	Nighthawk...						
☒	9141 Auction Type	Drop Down	OPEN,CL...						
☒	9142 Relative Limit Offset	Integer							
☒	9242 StartTime	DateTime							
☒	9243 Target Participation Rate	Number	1 <= Targe...						
☒	9244 Max Participation Rate	Number	1 <= Max ...						
☒	9245 Style	Drop Down	NORMAL...						
☒	9538 Custom Parameters	String							
☒	9574 Nighthawk Allocation	Number	1 <= Night...						
☒	User Entry Properties								
☒	IDEF_ACC Account	Account							
☒	IDEF_PRICE Pricing Instruction	Drop Down	Market, Li...						
☒	IDEF_LIFET Lifetime	Drop Down	End of Da...						
☒	IDEF_VT Number of ticks applied to volume ticks	Integer							
☒	IDEF_PT Tick price increment	Float							
☒	IDEF_LVL Override ladder volume step	Volume							
☒	IDEF_VOL Default order volume	Edit Drop Down							
☒	IDEF_MANAGE Manage order	Checkbox							
☒	IDEF_AUTOACK Auto acknowledge	Drop Down							
☒	IDEF_SUBDEST Sub Destination	Edit Drop Down							

This example code would set the External Algo Properties to 7000 and 9006 using the **ExecutionInstructions** input parameter after calling the *OrderCreate3* method:

// External Algo Properties are a subset of ExecutionInstructions and are supported in this field.

```
_OrderCreate3Input.Parameters.ExecutionInstructionsArray = new string[] { "7000(MyStrategy),9006  
(7:30:00 AM)"};
```

IPS

All the Web Services V4 samples linked in this document are available at this public GitHub repo: <https://github.com/iress/webservices-v4-docs>.



The samples include examples of SOAP XML requests and responses for key methods across each application, as well as a handful in different programming languages. To open a sample, click on the **Request** or **Response** column under the method you want to use.

ServiceSessionStart for IPS service	
 First, you will need to create an Iress session before creating a service session. You can use the SOAP request detailed here .	
Get IPS transactions (IPSTransactionGetByAccount5)	
Request	Response

IPS uploads

Please refer to the upload and API guide via the IPS status command as shown below for further information on Upload Specifications. The sample request below references a security list upload.

IPS Status

IPSName	Status	LoggedIn	Active	ProductVersion	Health	IOSPlusName
IPSSALESDEMO	Online	Yes	☐	19.11.97	OK	IPSNEO103_IOSP

Upload and API Guide - IPS 19.11.97

Upload Specifications

IPS supports bulk upload of data by various upload formats. The majority of these uploads support a variable number of fields. A header line must be provided which specifies the fields that are used in that upload file. The fields can be in any order. Mandatory fields must appear (sometimes a combination of fields) and all other fields can be excluded if desired. No footer is required for the files.

So, if for example you were doing a client detail upload, you must specify the AccountCode (to identify the account) and all other fields are optional. So if just wanted to specify the names:

IPUploadCreate1

[Request](#)

[Response](#)

IPUploadDataSet1

[Request](#)

[Response](#)

IPSUploadRun1	
Request	Response
IPSUploadSummaryGet2	
Request	Response
IPSUploadErrorGet1	
Request	Response

FIX+

All the Web Services V4 samples linked in this document are available at this public GitHub repo: <https://github.com/iress/web-services-v4-docs>.



The samples include examples of SOAP XML requests and responses for key methods across each application, as well as a handful in different programming languages. To open a sample, click on the **Request** or **Response** column under the method you want to use.

ServiceSessionStart for FIX+ service

First, you will need to create an Iress session before creating a service session. You can use the SOAP request detailed [here](#).

TargetIDGet

[Request](#)

[Response](#)

TargetIDStatusGet

This method checks if a FIX+ session is logged in:

FIXEngine	Description	Notes	Status	Num
FIXPLUSRECEIVEIOI			Online	1
FIXPLUSSENDIOI			Online	0

TargetID	LoggedIn	LastRecSeqNo	Last	TargetID	IOSName	LoggedIn	Online
IRERIOI	Yes	16		IRERIOI	AEDEMOP	Yes	Yes

[Request](#)

[Response](#)

Application Error Management

Go [here](#) for session error management.

Outside of session based error handling, documented in [Session error management](#), a calling application also needs to consider timeout and error handling at the method level.

When writing code to handle errors, be aware that some exceptions can originate within the client application (i.e. client-side web timeout), and some can be returned from the server.

There are different types of errors that can occur when using Web Services V4:

1. System errors

These generally return as SOAP faults (with HTTP status code 500) when making a method call, e.g. when a server has gone offline, the requesting user doesn't have the right permissions to call the method in question, the call timed out because the server took too long, etc

These also include timeouts from the Web server level and Iress application level.

Some examples include:

- Web server timeout :

"05-May 14:01:14.557 Request 'AuditTrailGetByUser' with ID '383fe5c4-c1b2-4d63-95ad-a131f0f232f2' has timed out while waiting for a response from server."

- Iress application timeout:

- Iress service:

""SQL request failed. Operation canceled?" on RequestID 90C324C7-FEA2-4394-BC7A-9939D9FB6979086EE43F-00C8-0014-0090-0005@IDSA01 from AdminUserCreate transaction invoked at 2011/05/02 15:39:56"

- IPS timeout:

"Database timeout occurred."

"Could not load data from database."

- **Data errors**

These are more typical on “Set” methods (eg. create, amend, delete, etc) particularly where the method supports bulk input, and the error information is returned in the response to indicate which inputs were successful and which failed. They are generally identified by inspecting the **ErrorNumber** and/or **ErrorMessage/ErrorDescription** output columns; in the case of a method supporting the **ErrorNumber** output column, a non-zero value indicates an error had occurred. For methods supporting **ErrorMessage** or **ErrorDescription**, a non-empty string typically indicates an error has occurred.

Some methods have both **ErrorNumber** and **ErrorMessage/ErrorDescription**.

- **Client-side errors**

These originate in the calling application, for example a client side web timeout:

“The request channel timed out while waiting for a reply after 00:01:00. Increase the timeout value passed to the call to Request or increase the SendTimeout value on the Binding. The time allotted to this operation may have been a portion of a longer timeout.”



Note: The way in which timeouts and errors need to be handled depends on the requirements of the calling client application; for example, in a GUI based application it may be preferable to show an error message to the end user so they can choose how to react; in the case of a batch or scheduled process, it may be preferable to implement a retry system with a wait mechanism in order to be as resilient as possible and ideally requiring no human interaction.



Tip: Error descriptions may change and it is recommended to avoid doing exact string matches to identify specific errors. Where possible, use the error number instead of an error message as typically the error number will not change.

 *If there is a requirement to handle recovery post handling any errors or timeouts, see [Application Recovery](#).*

Application Recovery

① *Session based recovery is documented [here](#).*

This topic covers where the calling application needs to handle recovery for specific methods, such as when receiving timeouts or errors on previous method calls.

The general principle for recovery with retrieval or “Get” methods, is to re-issue a new request once a new session has been established; whether it be an Iress session key for non-service based methods, or a Service session key for service based methods. Where a method supports paging bookmarks, it may be advantageous to recover from the last data record retrieved, instead of having to start the request right from the beginning.

For “Set” based methods such as those that create, amend, or delete data, recovery techniques may vary slightly across different categories of methods. In some cases, simply retrying the call may not be an option; for example when creating orders and sending an order to market, you would want to avoid the same order being placed twice. For other methods, it may be just a matter of retrying the call and handling any errors that occur.

The below section describes some specific recovery scenarios when using Iress WebAdmin methods, IOS+ Admin methods, as well as IOS+ orders for both retrieval and creation.

Iress WebAdmin methods

When using the Iress WebAdmin “set” methods (such as *AdminUserCreate2*, *AccountCreate*), where the **AmendIfExists** input parameter is available, we recommend setting the parameter to True. This can simplify recovery, as it allows the same method to be called again and a new record created if a matching record does not exist in the system.

If a matching record does exist, it will amend the record instead of returning an error.

Where **AmendIfExists** isn’t supported, we recommend checking what records had been successfully created previously by doing a retrieval first.

IOS+ admin methods

When using the IOS+ Admin “set” methods (such as *UserCreate*, *AccountCreate*), many support an **AmendIfExists** input parameter. We recommend setting the parameter to True. This can simplify recovery, as it allows the same method to be called again and a new record created if a matching record does not exist in the system.

If a matching record does exist, it will amend the record instead of returning an error.

Where **AmendIfExists** isn't supported, we recommend checking what records had been successfully created previously by doing a retrieval first.

IOS+ order retrieval

Retrieving orders along with live updates and handling order recovery.

Scenario: Recovery in scenarios where retrieving orders and order updates with a requirement to know about every order including those that did not go to market.

These recommendations apply to either application startup or recovery (e.g. network connection goes down, app crashes). Both should be handled in the same way. The aim is to get the best possible performance as well as reducing any potential data overlap between methods used to facilitate the recovery.

These recommendations are based upon the following:

- *OrderPadGet** methods are designed to support more narrow filtering and can be faster, particularly if looking at active orders only.
- *OrderSearchGet** methods support a wider set of filters but as a consequence are typically not as fast as using the *OrderPadGet** methods.
- Clients system(s) needing to reconcile orders, including those that have been created and deleted that did not go to market.

The recovery will be a two-step reconciliation process, where any order overlaps are handled to account for timing issues. The *GetByUser* version of the *OrderPadGetBy** and *OrderSearchGetBy** methods are described in the below for example, but the technique will apply to whatever primary filter used such as **Account**, **AccountGroup** etc.:

- Retrieve active orders only and start watching

Client application will first reconcile its orders against the active order set. Calling *OrderPadGetByUser* with *Updates=True* and *OrderFilter=5* ("All active orders") with a new unique *RequestID*. The client application should avoid requesting all orders today and yesterday (*OrderFilter=6*) or all orders today (*OrderFilter=7*), otherwise there will likely be a large overlap with orders that have already been processed throughout the day, especially if recovery needs to happen nearer market close. There may also be overlaps with orders returned in the next step (retrieving inactive orders only via *OrderSearchGetByUser*).

Long polling by continuously calling the *OrderPadGetByUpdates* method with the same *RequestID* in above *OrderPadGetByUser* data call. In order for the client application to be responsive during recovery and not waiting for the full reconciliation in steps 1 and 2 to complete, it can process any new orders received via the *OrderPadGetByUserUpdates* call immediately.

In the event that the snapshot is ahead of the updates call, client applications should handle any overlaps between the *OrderPadGetByUser* & *OrderPadGetByUserUpdates* calls.

For any request where you ask for a snapshot and updates:

1. Request starts watching first
 2. Snapshot request is performed
 3. Any updates that occur in between starting watching and getting the snapshot are sent to the web server.
 4. The web server buffers the updates
 5. Once the snapshot is retrieved, updates are played back and applied over the snapshot. These updates are not returned in the `OrderPadGetByUserUpdates` call.
- Retrieve any inactive orders that have been created/amended since the latest known order record

The calling application should then reconcile its own orders against inactive orders. This can be facilitated by calling the `OrderSearchGetByUser` method with the **DateTimeFrom** input parameter set to the latest known order record in the calling application, and the **OrderState** input parameter to 3 (inactive only).

The key here is to avoid getting active orders as well, because they are returned in the step 1 call to get all active orders using `OrderPadGetByUser`. Combined with the filtering to only get orders since the latest known order in the clients system, this would result in far less orders coming back than if they were to try and reconcile inactive orders as well via the `OrderPadGetByUser` method.

There is the potential for orders to go from active to inactive between step 1 and step 2. Clients will need to handle “double inactives”. For example, the `OrderPadGetByUserUpdates` call from Step 1 could see an order going from **ACTIVE** to **INACTIVE**, so if the same order is then retrieved in Step 2 via the **INACTIVE** only query they wouldn’t need to process that again. It could be that the update comes through first, or the record in `OrderSearchGetByUser` – so they could just do the same processing for whichever is first and throw away the second.

IOS+ order creation

When creating orders, it is recommended to populate the **OrderTag** input parameter with a unique ID, this may for example be the order reference in the clients own OMS system.

If orders are sent within a small time window (typically 5 minutes) that contain the same order tag, the IOS+ server will return an error indicating the order tag has already been used. This can prevent issues when orders are “double-sent”, as has been observed in some browsers historically.

In the event that a response to an order create is not received, a calling application can use the `OrderNoGetByOrderTag` method to lookup the IOS+ order number based on a

given order tag; when using this method, it is recommended to implement a retry system if a match is not initially found for the given order tag, given there may be delays on the order being committed into the database.

Client-side Optimisations and Server-side Limits

Client-side optimisations

Some recommendations for better performance:

1. Use the largest page size possible for each method being called, the more data a client app can receive per page means less overall requests. Each request has overheads, for example in terms of authentication and validation. By minimizing the number of requests this can lead to better performance.
2. Use “bulking” where available. E.g. if sending bulk data into the server and the method being used supports multiple input rows, send up to the maximum rows supported. As with point 1 above, the less requests being made to the server, the less overhead overall.
3. Ensure compression is enabled between both the client and server. While compression may consume a little more CPU on the web servers, it can result in a much faster transport time for requests, responses and updates. To enable compression set the HTTP header “Accept-Encoding” to “gzip”.
4. Ensure network speeds are not a bottleneck. For example if all computers in the network are on 1gbps speeds, but the calling computer is on 100mbps, speed improvements can be obtained by upgrading the calling computer’s network speed.
5. Use a “cut-down” WSDL based on the method filter feature; where only the methods required by the application are returned in the WSDL. We have seen a client application using the .NET Framework that used a full WSDL taking around 45 seconds to get a response to an IRESSSessionStart call, when that application was updated to use a cut down WSDL, this same method call dropped to under a second. Using a cut down WSDL can also improve overall responsiveness in development IDE, this has been observed with the Microsoft Visual Studio development environment.

Server-side limits

The following table describes the default limits setup on the web server that clients should become familiar with. These limits are in place to ensure optimal performance of the server for all users, by minimizing the impact that a single user can have on resources on the server. These are defaults but can be configured at the web server level if the client has their own local web server setup.

Limit	Value	Description
Max Page Size	1000	The maximum number of rows in a page to return to the client if the server does not specify for the method.
No update block time	1000	How many seconds to block waiting for updates to arrive from the server before returning an empty response to the client.
Request expiry time	1800	How many seconds an idle request (not being interacted with by the client) will remain active before it expires. The request will remain in memory for this length of time, and then it will be deleted if the client does not interact with the request within that time.
Request time out	25	The request timeout in seconds if the client specified no timeout.
Login Timeout	60	The session login timeout in seconds.
Max requests per session	50	The maximum number of active requests allowed per session.
Session expiry time	7200	How many seconds an idle session will remain active before it expires

Backward Compatibility

WSDL compatibility

In a client application there may be instances where there is a need to maintain multiple versions of WSDL references when upgrading to newer server versions.

Newer methods may become available, which provide an updated interface over a method exposed in previous versions. As we want to encourage usage of the newer interface the older versions may be removed from the new WSDL. In most cases, the older methods will still function in a calling client application, however, updating to the newer WSDL may result in compile errors for older methods still being used that are no longer in the new WSDL.

For example in IOS+, there are three versions of the *OrderCreate* method: *OrderCreate*, *OrderCreate2*, *OrderCreate3*. At the time of writing, a WSDL generated will only show *OrderCreate3* and not *OrderCreate* or *OrderCreate2*.

OrderCreate from an older WSDL

```
- <xsd:element name="OrderCreate">
  - <xsd:complexType>
    - <xsd:sequence>
      <xsd:element name="Input" type="tns:OrderCreateInput"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
- <xsd:element name="OrderCreateResponse">
  - <xsd:complexType>
    - <xsd:sequence>
      <xsd:element name="Output" type="tns:OrderCreateOutput" maxOccurs="1" minOccurs="0"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
- <xsd:complexType name="OrderCreateInput">
  - <xsd:sequence>
    <xsd:element name="Header" type="tns:OrderCreateInputHeader" maxOccurs="1" minOccurs="1" nillable="true"/>
    <xsd:element name="Parameters" type="tns:OrderCreateInputParameters" maxOccurs="1" minOccurs="1" nillable="true"/>
    <xsd:any maxOccurs="unbounded" minOccurs="0" processContents="lax"/>
  </xsd:sequence>
</xsd:complexType>
- <xsd:complexType name="OrderCreateInputHeader">
  - <xsd:sequence>
    <xsd:element name="ServiceSessionKey" type="xsd:string" maxOccurs="1" minOccurs="1" nillable="true"/>
    <xsd:element name="RequestID" type="xsd:string" maxOccurs="1" minOccurs="1" nillable="true"/>
    <xsd:element name="Updates" type="xsd:boolean" maxOccurs="1" minOccurs="1" nillable="true" default="false"/>
    <xsd:element name="Timeout" type="xsd:int" maxOccurs="1" minOccurs="1" nillable="true" default="25"/>
    <xsd:element name="PageSize" type="xsd:int" maxOccurs="1" minOccurs="1" nillable="true" default="0"/>
    <xsd:element name="WaitForResponse" type="xsd:boolean" maxOccurs="1" minOccurs="1" nillable="true" default="true"/>
    <xsd:element name="PagingBookmark" type="tns:OrderCreatePagingBookmark" maxOccurs="1" minOccurs="1" nillable="true"/>
    <xsd:element name="PagingDirection" type="xsd:int" maxOccurs="1" minOccurs="1" nillable="true" default="0"/>
    <xsd:any maxOccurs="unbounded" minOccurs="0" processContents="lax"/>
  </xsd:sequence>
</xsd:complexType>
- <xsd:complexType name="OrderCreatePagingBookmark">
  - <xsd:sequence>
    <xsd:any maxOccurs="unbounded" minOccurs="0" processContents="lax"/>
  </xsd:sequence>
</xsd:complexType>
- <xsd:complexType name="OrderCreateInputParameters">
  - <xsd:sequence>
    <xsd:element name="BuyOrSellArray" maxOccurs="1" minOccurs="1" nillable="true">
      - <xsd:complexType>
        - <xsd:sequence>
          <xsd:element name="BuyOrSell" type="xsd:string" maxOccurs="unbounded" minOccurs="0" nillable="true"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>
  </xsd:sequence>
</xsd:complexType>
```

OrderCreate from a newer WSDL

```

▼<xsd:element name="OrderCreate3">
  ▼<xsd:complexType>
    ▼<xsd:sequence>
      <xsd:element name="Input" type="tns:OrderCreate3Input"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
▼<xsd:element name="OrderCreate3Response">
  ▼<xsd:complexType>
    ▼<xsd:sequence>
      <xsd:element minOccurs="0" maxOccurs="1" name="Output" type="tns:OrderCreate3Output"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
▼<xsd:complexType name="OrderCreate3Input">
  ▼<xsd:sequence>
    <xsd:element minOccurs="1" maxOccurs="1" name="Header" type="tns:OrderCreate3InputHeader" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="Parameters" type="tns:OrderCreate3InputParameters" nillable="true"/>
    <xsd:any processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
  </xsd:sequence>
</xsd:complexType>
▼<xsd:complexType name="OrderCreate3InputHeader">
  ▼<xsd:sequence>
    <xsd:element minOccurs="1" maxOccurs="1" name="ServiceSessionKey" type="xsd:string" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="RequestID" type="xsd:string" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="Updates" type="xsd:boolean" default="false" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="Timeout" type="xsd:int" default="25" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="PageSize" type="xsd:int" default="0" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="WaitForResponse" type="xsd:boolean" default="true" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="PagingBookmark" type="tns:OrderCreate3PagingBookmark" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="PagingDirection" type="xsd:int" default="0" nillable="true"/>
    <xsd:element minOccurs="1" maxOccurs="1" name="InputLocalizationType" type="xsd:int" default="0" nillable="true"/>
    <xsd:element minOccurs="0" maxOccurs="1" name="OutputLocalizationType" type="xsd:int" default="0" nillable="true"/>
    <xsd:any processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
  </xsd:sequence>
</xsd:complexType>
▼<xsd:complexType name="OrderCreate3PagingBookmark">
  ▼<xsd:sequence>
    <xsd:any processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
  </xsd:sequence>
</xsd:complexType>
▼<xsd:complexType name="OrderCreate3InputParameters">
  ▼<xsd:sequence>
    ▼<xsd:element name="ParentOrderNumberArray" nillable="true" minOccurs="1" maxOccurs="1">
      ▼<xsd:complexType>
        ▼<xsd:sequence>
          <xsd:element name="ParentOrderNumber" type="xsd:long" nillable="true" minOccurs="0" maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>
    ▼<xsd:element name="SideCodeArray" nillable="true" minOccurs="1" maxOccurs="1">
      ▼<xsd:complexType>
        ▼<xsd:sequence>
          <xsd:element name="SideCode" type="xsd:string" nillable="true" minOccurs="0" maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>
    ▼<xsd:element name="AccountCodeArray" nillable="true" minOccurs="1" maxOccurs="1">
      ▼<xsd:complexType>
        ▼<xsd:sequence>
          <xsd:element name="AccountCode" type="xsd:string" nillable="true" minOccurs="0" maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>
  </xsd:sequence>
</xsd:complexType>

```



Note: The list of methods available against a service may vary not only between versions but also based on server configuration. For example, on an IOS+ server additional methods will be returned in the WSDL when the IOS+ Retail Server module is installed



Tip: We strongly recommend that clients save a copy of the raw WSDLs for each of the versions used in their projects. When a new WSDL is obtained it should be compared with previous versions to determine the exact differences in the interface and can be used to determine how the changes may impact a client application. A great way to compare the differences are by using a tool such as BeyondCompare, Winmerge or VS Code.

Appendix

Possible session error codes and causes

Note, this list is not exhaustive and new errors may be added over time. In addition, there may be some changes to error descriptions so it is not recommended to do exact string matches.

Error number	Error description	Action to be taken when encountering the error in code	Action to take when investigating the error
0	Login failed, awaiting login response	Retry the request again with a new unique RequestID	No action, possibly caused by slowness on the server
25001	No RequestID was specified in the SOAP request parameters	Generate a new unique RequestID and retry the request again	Make sure that a non-empty RequestID string is always specified in all SOAP requests
25002	The specified RequestID was not found on the server	Can be ignored	When making a change to an existing request (e.g. ending a request with SessionRequestEnd) make sure to specify a valid existing RequestID
25003	The request with the specified RequestID has already ended	Can be ignored	Once a request has ended, do not attempt to end it again
25004	The request failed to be parsed correctly	Fail gracefully, there's no recovery if the SOAP request is invalid	Make sure that the request is valid (e.g. method name is valid, elements are valid, etc.)

Error number	Error description	Action to be taken when encountering the error in code	Action to take when investigating the error
25006	The service session referenced in the request is no longer valid	Create a new service session and retry the request against the new service session	Do not end the service session when further requests are still going to be called against it. This error can also occur if the server (e.g. IOS+) goes offline and the service sessions against the server are invalidated. Or if an IOS+ group access setting changed.
25007	Maximum number of queued update rows has been reached (default 10,000 rows), the request has been forcibly ended	Retry the request again with a new unique RequestID	Make sure to retrieve updates as fast as possible, to avoid filling up the server side cache
25008	Server error, currently used for login failures but may be used for others in the future	Fail gracefully, there's no recovery if the login details or user permissions are incorrect	Check that the login details in the request are correct and that the user has the appropriate permissions
25009	Invalid service session key specified	Create a new service session and retry the request against the new service session	Check that the service session key specified in the SOAP request is correct and that the service session was not previously ended
25010	The bookmarks specified in the request failed to parse	Fail gracefully	Check that the bookmarks specified in the SOAP request is correct (e.g. types are correct and within valid ranges)
25011	Not allowed to create multiple service sessions to the same server	Retry the request again with a new unique RequestID	No action, inform your Iress Account Executive or support desk with details of the error
25012	Invalid server name specified	Fail gracefully	Check that the server name specified is valid

Error number	Error description	Action to be taken when encountering the error in code	Action to take when investigating the error
25013	No session key was provided in request	Create a new Iress session receive a new IRESSSessionKey	Check that all SOAP requests contain a valid session key
25014	Invalid session key specified	Create a new Iress session receive a new IRESSSessionKey	Check that the session key specified in the SOAP request is valid
25015	Internal error, server is offline	Retry the request again with a new unique RequestID	No action, inform your Iress Account Executive or support desk with details of the error
25016	Maximum number of active sessions reached (default 20,000 sessions), no more session can be created	Fail gracefully	Either reduce the total number of sessions used or once the maximum number of active sessions limit is reached, end some existing Iress sessions before attempting to start any new Iress sessions
25017	Internal session allocation error	Fail gracefully	No action, inform your Iress Account Executive or support desk with details of the error
25018	Impersonation is not allowed on this WebServices instance	Fail gracefully	Do not specify an IRESSSessionKey in the header of the IRESSSessionStart request
25019	Invalid IRESSSessionKey or service session key specified	Create a new Iress session or service session to receive a new session key	Check that the IRESSSessionKey or service session key specified is valid. This error can also occur if the server (e.g. IOS+) goes offline and the service sessions against the server are invalidated. Or if an IOS+ group access setting changed.

Error number	Error description	Action to be taken when encountering the error in code	Action to take when investigating the error
25020	Login failed	Fail gracefully	Check that the login details in the request are correct and that the user has the appropriate permissions
25021	Internal error, request is in use	Retry the request again with a new unique RequestID	No action, inform your Iress Account Executive or support desk with details of the error
25022	Request cannot be made because user is not logged in	Create a new Iress session and retry	The Iress session ended when a new request against that session was made. Either create a new Iress session before making the request or ensure that the Iress session is not ended when the request is made.
25023	Internal error, RequestID not found	Retry the request again with a new unique RequestID	No action, inform your Iress Account Executive or support desk with details of the error
25024	The specified RequestID was for a different request type	Retry the request again with a new unique RequestID	Check that RequestIDs of any active requests are not being reused for any other requests
25025	The request is no longer watching for updates	Retry the request again with a new unique RequestID	Check that a RequestID that has been ended is not being used again to retrieve update data
25026	Maximum number of active requests for the session has been reached, no more new requests can be made	Fail gracefully	End any active requests in the session before making new requests
25027	The request with the specified RequestID has already finished	Retry the request again with a new unique RequestID	Do not reuse a RequestID that was previously used. Generate a new unique RequestID for any new requests

Error number	Error description	Action to be taken when encountering the error in code	Action to take when investigating the error
25028	The request with the specified RequestID has expired	Retry the request again with a new unique RequestID	An idle request (e.g. a request watching updates but never retrieving the update data) will expire after a period (default 30 mins). Keep the request active to avoid expiration
25031	The request was terminated because of a session logout request	Fail gracefully or create a new Iress session and retry	Make sure that the session is not ended while a request is still in progress
25032	The request timed out while waiting for a response from the server	Action would depend on the type of request. E.g. if it was a read-only request for data, then retry the request with a new unique RequestID, or if it was a request that modifies data, check the state of the data	
25033	The service session was terminated	Create a new service session and retry the request against the new service session	The referenced service session was terminated. The error message contains the reason for termination (e.g. IOS+ group access changed) which may provide guidance on what action needs to be taken
25035	The server is offline	Create a new service session and retry the request against the new service session	No action, inform your Iress Account Executive or support desk with details of the error
666	Invalid session key specified, invalid SOAP request (e.g. XML contains invalid elements)	Fail gracefully	No action, inform your Iress Account Executive or support desk with details of the error

Error number	Error description	Action to be taken when encountering the error in code	Action to take when investigating the error
666	Could not communicate with data server...	Create a new Iress session and retry	No action, inform your Iress Account Executive or support desk with details of the error. It is possible that this was a scheduled outage (e.g. software upgrade)
666	Could not located IDS for this request	Create a new Iress session and retry	No action, inform your Iress Account Executive or support desk with details of the error. It is possible that this was an outage or session key mixup. E.g. the request passing a session key with ...@IDSA32 but there is no such IDS on the cluster

Support queries

When raising a support query with Iress, please ensure the following information is logged and provided:

1. Error message received from Iress

With an error message, Iress can quickly determine the source of the error and the actions that need to be taken to resolve

3. Name of method called

Knowing the name of the method, allows Iress to quickly direct the query to the appropriate team to investigate

5. Date and time incident occurred

An accurate timestamp, allows Iress to quickly narrow down the window of investigation

7. RequestID

With a unique RequestID, Iress can reconcile the path of the request through the Iress backend, which allows us to figure out what happened to it

9. IressSessionKey and/or ServiceSessionKey

Having the session key(s) allows Iress to investigate the state of the session (e.g. whether it is logged in/out, the active requests, etc.)

11. Username@company

With the username@company, Iress can check that the user has the appropriate permissions and the sessions the user is logged in to

13. Service (e.g IPS/IOS+/Iress)

Knowing the service being used allows Iress to quickly direct the query to the appropriate team to investigate

15. Instance name (IPS name, IOS+ name)

The instance name allows Iress to narrow down the scope of the investigation to just the server in question

17. Request details (input parameters)

The input parameters used allows Iress to quickly determine whether the request had valid parameters (e.g. the date range provided may be beyond the configured limit)

19. WebServices endpoint used

This allows Iress to narrow the scope of the investigation to the servers servicing the particular endpoint

 Please provide as much of the above information as possible. The more information we have, the faster we can troubleshoot.

Frequently Asked Questions

Request has exceeded the maximum of 10000 queued updates

The web server has a limit of 10000 queued updates for each **RequestID**. Reaching this limit suggests that the calling application is not polling updates frequently enough. We recommend polling at a higher frequency to avoid this. If higher frequency polling does not resolve the issue we recommend splitting up the request into multiple smaller requests with different filters.

As an example, when watching orders for a list of accounts via *Order-PadGetByAccountGroupUpdates*. Instead of one request with a large account group you can split up the Account group into smaller groups and have one request for each account group.

The session has reached the limit of 50 active requests. You cannot create any more requests

The calling application needs to ensure:

- Data for each request should be polled until the state is no longer data pending. This applies both when using asynchronous requests, and/or paging. This will ensure that the request does not remain active.
- In the scenario where a client only needs a partial result set, and doesn't want to continue polling until the state is no longer pending, the other option is to call *SessionRequestEnd* and specify the specific **RequestID** to end.
- In the scenario where the calling application is calling many different types of methods concurrently, this limit may be more easily hit. For example, a buy side client connecting to many different sell side IOS+ servers concurrently and calling methods on each on a single *IRESSSessionKey*. It would be beneficial to either split up the requests across multiple sessions, or make the requests more sequential

Do you support compression?

Yes, we support HTTP compression via gzip. Please set the HTTP header "Accept-Encoding" to "gzip" in the request. Please consult your development toolkit on how this header can be passed.

If compression is also enabled on the server, the response header will contain “Content-Encoding: gzip” indicating compression is successfully activated.

API Method Quick Reference

Iress Pro

For more details see [Iress Pro - Market Data and Web Admin](#)

IRESSSessionStart	
Request	Response
TimeSeriesGet2	
Request	Response
PricingQuoteGet	
Request	Response

IOS+

For more details see [IOS+](#)

ServiceSessionStart for IOS+ service	
Request	Response
Create order (OrderCreate3)	
Request	Response
Amend order (OrderAmend2)	
Simple volume amend.	
Request	Response
Cancel order (OrderDelete)	
Request	Response

OrderPadGetByAccount and OrderPadGetByAccountUpdates

OrderPadGetByAccount or other *OrderPadGetBy** methods are used to get a snapshot of the order pad and can be used in conjunction with their own updates method to receive ongoing updates for the orders received or new orders as well. If you are intending to watch for updates please set the updates parameter to true for both methods

OrderPadGetByAccount: [Request](#)

OrderPadGetByAccount: [Response](#)

OrderPadGetByAccountUpdates: [Request](#)

OrderPadGetByAccountUpdates: [Response](#)

 You need to pass the same **RequestID** for updates method which you used for *OrderPadGetByAccount* method.

Get bookings (BookingGetByOrganisation2)

[Request](#)

[Response](#)

Get ETCs (ETCGetByOrganisation)

[Request](#)

[Response](#)

IOS+ Contingent Orders

Order placement with take profit

[Request](#)

[Response](#)

FIX+

 For more details see [FIX_Plus](#)

ServiceSessionStart for FIX+ service

 First, you will need to create an Iress session before creating a service session. You can use the SOAP request detailed [here](#).

TargetIDGet

[Request](#)

[Response](#)

TargetIDStatusGet

[Request](#)

[Response](#)

IPS

 For more details see [IPS](#)

ServiceSessionStart for IPS service	
 First, you will need to create an Iress session before creating a service session. You can use the SOAP request detailed here .	
Get IPS transactions (IPSTransactionGetByAccount5)	
Request	Response
IPSUploadCreate1	
Request	Response
IPSUploadDataSet1	
Request	Response
IPSUploadRun1	
Request	Response
IPSUploadSummaryGet2	
Request	Response
IPSUploadErrorGet1	
Request	Response



Offices in:
Australia, Canada, Hong
Kong, New Zealand
Singapore, South Africa,
United Kingdom.